



Projeto 2: Conectar Banco de Dados com P-O-O

LibriTech

CESED/UNIFACISA

Sistemas de Informação

Competência:

Conectar Banco de

Dados com P-O-O

2026-02

GRUPO:

Arthur Pereira Silva

Bernardo Ramos dos Santos

Rodrigo Lira Rodrigues

Luiz Gustavo Barbosa Machado

Professora:

Anna Beatriz

Lucena Lira

1.Introdução:

Este projeto apresenta o desenvolvimento do LibriTech, um sistema de gestão de uma biblioteca universitária. A proposta foi construir um sistema em que o MySQL atua como o "cérebro" que protege as regras de negócio e a segurança, enquanto o Java funciona como a interface que conversa com esse banco. A ideia central é que a segurança seja imposta pelo próprio banco de dados, e não apenas pela interface gráfica.

2.Objetivos:

O objetivo principal foi aplicar na prática, de forma integrada, todo o conteúdo da disciplina: **Views** para consultas estruturadas, **Stored Procedures** para encapsular regras de negócio em transações, **Triggers** para automação e auditoria, **Índices** para otimização, e um sistema de **usuários e permissões** (GRANT/REVOKE) seguindo o princípio do privilégio mínimo.

Além disso, conectamos o banco a uma aplicação **Java orientada a objetos** via **JDBC**, com login real e menus dinâmicos.

3.Arquitetura do Sistema:

O sistema une dois mundos, o **Java** (orientado a objetos) e o **MySQL** (relacional) conectados pela ponte do **JDBC** que permite enviar comandos SQL direto do Java. A aplicação é dividida em camadas **Model** (classes de POO), **DAO** (acesso a dados), **Conexão e Banco** e a interface com o usuário é feita com **JOptionPane**. E cada operação roda com as permissões do usuário que fez login.

4.Estrutura do Banco (DDL):

O banco tem **6 tabelas ligadas por chaves primárias e estrangeiras (Usuarios, Enderecos, Livros, Emprestimos, Multas e Log_Auditoria)**, que garantem a integridade dos dados. Seguimos boas práticas: endereço em tabela separada (**1ª Forma Normal**), **DECIMAL** para dinheiro e **ENUM** para o tipo de usuário.

5.POO no Java:

A camada Java usa os três pilares da orientação a objetos: **Encapsulamento** (atributos privados com validação), **Herança** (a classe Usuario que origina Aluno e Funcionario) e Polimorfismo que seria o cálculo de prazo retorna 7 dias para aluno e 14 para funcionário.

6.Segurança e Privilégio Mínimo:

Foram criados **4 usuários** no MySQL, cada um com permissões definidas por **GRANT/REVOKE**: o **gerente** tem acesso total e faz backup, o **bibliotecário** opera o dia a dia, mas não apaga a auditoria; o **estagiário** tem o mínimo, com o DELETE explicitamente revogado e o **aluno** só tem leitura, e apenas por uma View. No login, o Java conecta usando as credenciais digitadas ou seja, **as permissões dependem de quem logou no banco.**

7.Observação/armadilha em Estagiário:

Todo funcionário vê o **mesmo menu** e não escondemos botões. Quando o estagiário tenta excluir um livro, o **banco bloqueia** (pelo REVOKE DELETE) e o Java mostra "**Acesso Negado**". Prova de que a segurança está no **banco**, e não na tela.

8.Stored Procedures:

As procedures encapsulam as regras de negócio em transações. A de **empréstimo** verifica pendências e estoque, registra o empréstimo e dá baixa no estoque e, se qualquer passo falha, executa **ROLLBACK**.

Já a de **cadastro** insere usuário e endereço numa única transação: se o endereço falha, o usuário também é desfeito, garantindo a atomicidade e evitando registros órfãos.

9.Triggers:

Os gatilhos automatizam regras e segurança. A trigger de **horário comercial** bloqueia operações fora das 08h–18h, a de **auditoria** registra no Log todo livro excluído, salvando os dados antigos em **JSON** e quem realizou a ação e a de **limite** impede que um aluno com 3 livros pegue um quarto e a de estoque impede que a quantidade fique negativa.

10.Views:

As views organizam e protegem o acesso aos dados. A **acervo público** mostra o catálogo ao aluno escondendo o preço de custo, a de **livros atrasados** lista os empréstimos vencidos com o contato do usuário, a do **ranking** traz o top 10 de livros mais emprestados e a do **dashboard** resume o total de multas arrecadadas e pendentes.

11. Análise de Performance (Índices e EXPLAIN):

Para provar a otimização, criamos um índice na coluna **titulo** da tabela Livros e comparamos o plano de execução com o comando **EXPLAIN**. Antes do índice, a coluna type mostrava **ALL** (varredura completa), lendo cerca de 2010 linhas. **Depois**, passou para **ref**, usando o índice e lendo apenas **1 linha**.

Antes

Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Livros	NULL	ALL	NULL	NULL	NULL	NULL	2010	10.00	Using where

Depois

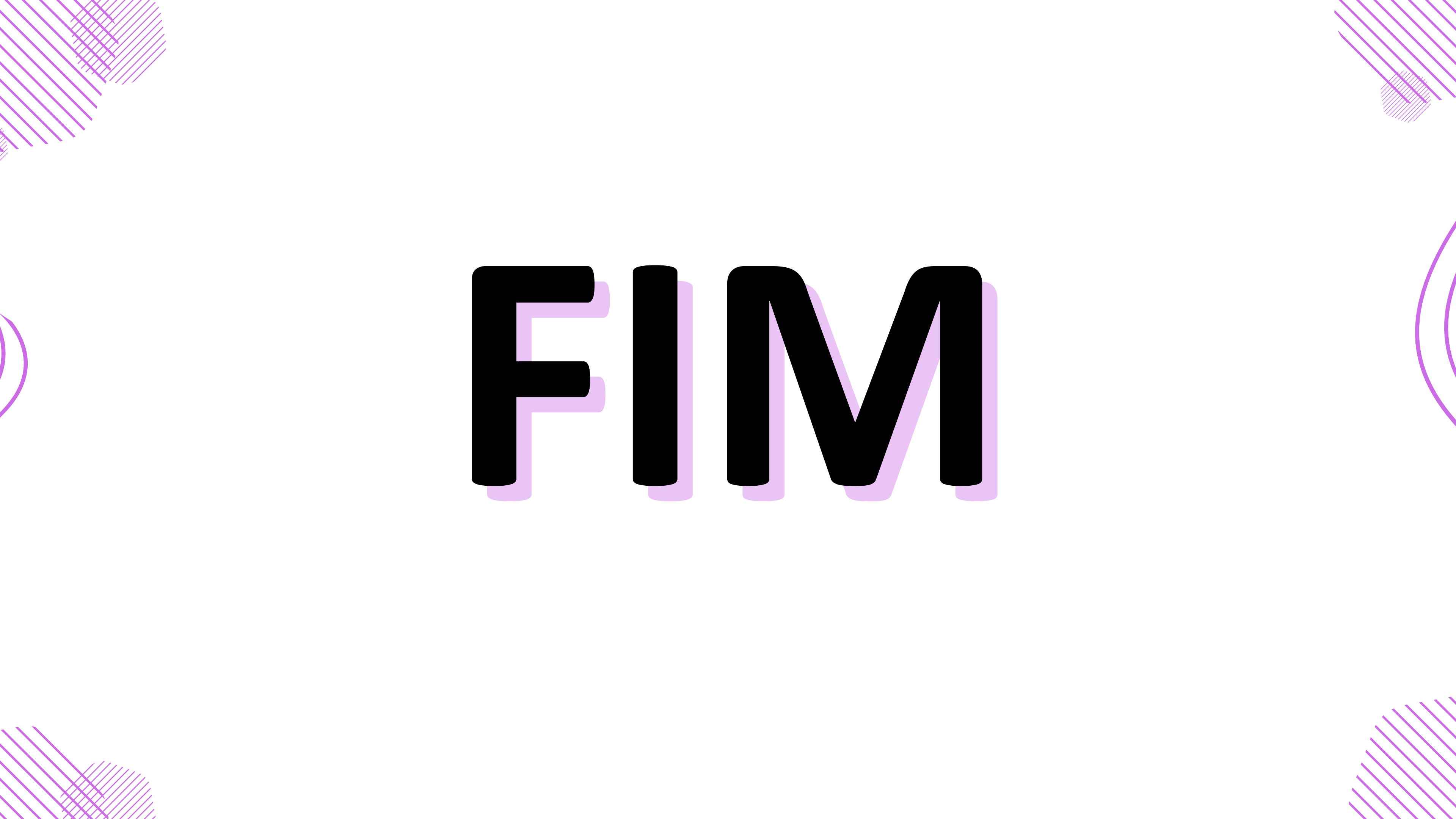
Result Grid Filter Rows: Export: Wrap Cell Content:												
	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
▶	1	SIMPLE	Livros	NULL	ref	idx_livro_titulo	idx_livro_titulo	802	const	1	100.00	NULL

12. Plano de Testes:

- **Horário:** operação fora do horário (8h-18h) → erro de trigger
- **Atomicidade:** cadastro com endereço inválido → usuário não é salvo
- **Segurança:** estagiário tenta excluir → bloqueado
- **Auditoria:** gerente exclui → registra no Log
- **Limite:** 4º empréstimo do mesmo aluno → bloqueado
- **Multa:** devolução atrasada, há o calculo do valor certo → valor correto
- **View:** aluno não consegue ver o preço de custo
- **EXPLAIN:** comprovação antes/depois do índice, o quê mostra otimização

13. Conclusão:

O LibriTech entregou um sistema completo, unindo um **banco de dados robusto** a uma **aplicação Java integrada**. A segurança foi garantida pelo próprio banco (privilégio mínimo, triggers e views), a **integridade** pelas transações e chaves estrangeiras, e a **performance** pelos índices. O projeto consolidou, na prática, a **ponte entre a Programação Orientada a Objetos e o Banco de Dados Relacional**.

The image features a white background with decorative purple line patterns in the corners. The top-left and bottom-left corners have dense, overlapping diagonal lines. The top-right and bottom-right corners have more sparse, parallel diagonal lines. The word "FIM" is centered in a bold, black, sans-serif font. Each letter has a light purple shadow offset to its right and bottom, giving it a 3D appearance.

FIM